Reviewer Pack — Minimal p-Adic Lp-Norm and Circuit Monotone Width Inequality

Entry d3d45d17f231 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 04:23:37 UTC

Minimal p-Adic Lp-Norm and Circuit Monotone Width Inequality

Entry ID: `d3d45d17f231` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-06 04:23:37 UTC

1. Conjecture

Field A (mathematical branch): p-adic Analysis **Field B** (complexity object): Boolean Circuit Complexity

Statement:

> For every CNF φ with n variables, the minimal p-adic Lp-norm of its clause indicator polynomial (defined as the smallest value of |φ_j|^p for some clause j), denoted as min(Lp(φ)), is linearly correlated with its circuit monotone width w_mw(φ), such that min(Lp(φ)) = Θ(w_mw(φ)). Equivalently, for all CNFs φ, Lp(φ) ≤ C * w_mw(φ)^p for some constant C > 0.

Rationale (proposer's reasoning):

> The p-adic Lp-norm captures the 'sparsity' of a polynomial in p-adic numbers, which may reflect the structural complexity of its zeros or roots. This could provide insights into the hardness of satisfiability for CNFs by relating it to circuit monotone width, a measure of the complexity of circuits that can recognize the language defined by a CNF.

Taxonomy category: `algorithmic_complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `0a3e360afc97357d`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a CNF with n variables, if the Pearson's correlation coefficient between min(Lp(φ)) and w_mw(φ) exceeds 0.8 and the mean of Lp(φ) values across seeds is less than or equal to 3, then support for the conjecture is provided.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal p-adic Lp-Norm" AND "Boolean circuit monotone width" - "clause indicator polynomial" AND "circuit monotone width" - "p-adic analysis" IN title AND "Boolean circuit complexity" IN title

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        cnf = []
        for _ in range(n):
            clause = [random.randint(1, n), -random.randint(1, n)]
            cnf.append(clause)
        return cnf

    def clause_indicator_polynomial(cnf):
        n = len(cnf)
        indicator = [0] * (2**n)
        for i in range(2**n):
            if all((i & (1 << j)) != 0 for j in range(n) if cnf[j][0] == -j-1 or cnf[j][1] == j+1)
                indicator[i] = 1
        return indicator

    def p_adic_l_p_norm(indicator, p):
        norm = sum(abs(x)**p for x in indicator)
        return norm**(1/p) if norm > 0 else 0

    def monotone_gadget(cnf):
        n = len(cnf)
        gadget = []
        for i in range(2**n):
            clause = [0] * (2**n)
            for j in range(n):
                if cnf[j][0] == -j-1 and (i & (1 << j)) != 0:
```

```
                    clause[i ^ (1 << j)] = 1
                elif cnf[j][1] == j+1 and not (i & (1 << j)):
                    clause[i ^ (1 << j)] = 1
            gadget.append(clause)
        return gadget

    def monotone_width(gadget):
        n = len(gadget)
        width = 0
        for i in range(2**n):
            if all((i & (1 << j)) != 0 for j in range(n) if gadget[j][i] == 1):
                width += 1
        return width

    def pearson_correlation(x, y):
        n = len(x)
        mean_x = sum(x) / n
        mean_y = sum(y) / n
        cov_xy = sum((x[i] - mean_x) * (y[i] - mean_y) for i in range(n)) / n
        var_x = sum((x[i] - mean_x)**2 for i in range(n)) / n
        var_y = sum((y[i] - mean_y)**2 for i in range(n)) / n
        return cov_xy / (math.sqrt(var_x) * math.sqrt(var_y))

    n_values = [5, 10, 15, 20, 30, 40]
    l_p_norms = []
    widths = []

    for n in n_values:
        cnf = generate_cnf(n)
        indicator = clause_indicator_polynomial(cnf)
        l_p_norm = p_adic_l_p_norm(indicator, 2)  # Using L2 norm for simplicity
        l_p_norms.append(l_p_norm)

        gadget = monotone_gadget(cnf)
        width = monotone_width(gadget)
        widths.append(width)

    correlation = pearson_correlation(l_p_norms, widths)
    mean_l_p_norm = sum(l_p_norms) / len(l_p_norms)

    return {
        "metric_name": "Pearson's Correlation Coefficient",
        "metric_value": correlation,
        "instances_tested": 6,
        "n_max": max(n_values),
        "conjecture_holds": correlation > 0.8 and mean_l_p_norm <= 3,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
```

```
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_hol
        print(f"RESULT: FALSIFIED counterexample='' first_failing_seed={first_failing_seed}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0f0ab21f.py", line 102, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0f0ab21f.py", line 81, in run_trial
    width = monotone_width(gadget)
            ^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0f0ab21f.py", line 57, in monotone_widt
    if all((i & (1 << j)) != 0 for j in range(n) if gadget[j][i] == 1):
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0f0ab21f.py", line 57, in <genexpr>
    if all((i & (1 << j)) != 0 for j in range(n) if gadget[j][i] == 1):
                                                    ~~~~~~~~~~^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means that the pre-registered support conditions could not be evaluated. | next: Investigate and fix the crash in the test code to allow for proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12481
2	propose	ollama_remote	glm4:latest	0	0	12500
3	propose	ollama_remote	glm4:latest	0	0	13308
4	propose	ollama_remote	glm4:latest	0	0	21777
5	preregistration	ollama_remote	glm4:latest	0	0	9441
6	novelty	ollama_remote	glm4:latest	0	0	8365
7	novelty	ollama_remote	glm4:latest	0	0	16235
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16232
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	38024
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24658
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17036
12	judge	ollama_remote	glm4:latest	0	0	62880

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 252935 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in `research/audit/d3d45d17f231.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d3d45d17f231.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/d3d45d17f231.tar.gz` (if generated)